

PRD for Enterprise Retail POS & Inventory Management Platform

1. Context & Problem Statement

The business currently operates across fragmented systems for retail sales, inventory management, and financial operations. This fragmentation results in:

- Manual reconciliation across systems
- Inconsistent inventory and financial records
- Limited visibility into stock movement across branches
- Delayed decision-making due to lack of real-time data
- High operational overhead

There is a need for a single, integrated platform that consolidates operations, enforces financial integrity, and enables real-time decision-making across multiple locations.

2. Product Goals

Primary Goals

- Unify retail, inventory, and finance workflows into a single system
- Ensure financial correctness and auditability by design
- Reduce manual processes across inventory and accounting by $\geq 50\%$
- Enable real-time stock visibility across all branches

Secondary Goals

- Improve customer tracking and engagement
- Introduce intelligent automation (OCR, anomaly detection)
- Establish a scalable foundation for multi-location expansion

3. Non-Goals

- Building a fully customizable accounting platform comparable to enterprise ERPs
- Supporting external marketplace integrations in initial phases
- Native mobile applications (web-first delivery)

4. Product Scope & Phasing

Phase 1 (MVP – Revenue & Operations Critical)

- POS system
- Inventory management (multi-branch)
- CRM
- Stock movement tracking
- Inter-branch requisition (basic flow)

Phase 2 (Financial Control & Logistics)

- Accounting engine (GL, AP, AR)
- Tax handling (VAT/GST)
- Financial reporting
- Advanced requisition approvals (MD-level)
- Backend demand & fulfillment system

Phase 3 (Optimization Layer)

- OCR processing
- AI-driven anomaly detection
- Natural language analytics
- Predictive inventory alerts

5. Users & Permissions Model

Roles

- Cashier: Executes transactions
- Store Keeper: Manages inventory and requisitions
- Accountant: Oversees financial records and reporting
- MD/Admin: Approval authority and full system visibility

Requirements

- Strict RBAC enforcement at API and UI levels
- All actions must be attributable to a user (audit requirement)

6. Core Product Requirements

6.1 POS & Transaction Engine

Capabilities

- Mixed-cart transactions (retail items)
- Multiple payment methods (cash, transfer, card)

- Split payments and partial payments
- Layby and reservation workflows

System Requirements

- Transactions must be atomic and idempotent
- Each transaction generates:
 - Inventory updates
 - Accounting entries (Phase 2)
- Offline transaction capture with eventual sync

6.2 Inventory & Multi-Branch Stock Management

Capabilities

- Centralized inventory across all branches
- Real-time stock visibility
- Stock movement tracking (sales, transfers, adjustments)

Key Requirement

Every stock removal must be:

- Recorded
- Visible system-wide
- Attributable to a user and action

Inventory Events

- Sale
- Transfer
- Adjustment
- Requisition fulfillment

Constraints

- Inventory updates must be event-driven
- No silent stock changes allowed

6.3 Inter-Branch Requisition Workflow

Overview

Branches can request stock from other branches or a central warehouse through a structured approval workflow.

Workflow

1. Requisition Initiation

- Initiated from backend (CRM/Admin panel)
- Store Keeper raises request:
 - Item(s)
 - Quantity
 - Destination branch

2. Approval Process

- Routed to MD/Admin for approval
- Approval required before fulfillment

3. Fulfillment

- Approved request triggers:
 - Inventory reservation at source
 - Stock deduction upon dispatch
 - Shipment preparation

4. Delivery & Confirmation

- Receiving branch confirms receipt
- Inventory updated accordingly

Status Tracking

- Pending
- Approved
- Shipped
- Delivered

6.4 Backend Demand & Fulfillment System

Capabilities

- Branches can initiate stock demand directly from backend
- Centralized visibility of all requests

Flow

1. Demand created
2. Routed for approval
3. Approved demand triggers:
 - Inventory allocation
 - Shipment process
4. Items shipped to requesting branch

6.5 Stock Visibility & Movement Tracking

Requirement

Anytime stock is removed, it must be visible across the system.

System Behavior

- Real-time updates across all branches
- Full movement logs:
 - Who performed the action
 - When it occurred
 - Why (sale, transfer, adjustment, requisition)

Audit Capability

- Immutable logs of all stock movements
- Traceability for every unit of inventory

6.6 Low Stock Alerts & Trigger System

Trigger Condition

- Stock falls below predefined threshold

Notifications

- Managing Director (MD)
- Store Keeper
- Relevant branch personnel

Delivery Channels

- In-app notifications
- Email (future phase)

Outcome

- Proactive replenishment
- Reduced stockouts

6.7 Accounting Engine (Phase 2)

Capabilities

- Double-entry bookkeeping system
- Chart of Accounts management
- Journal entry generation from system events

System Requirements

- Every transaction produces:
 - Debit entry
 - Credit entry
- No orphaned entries allowed

Reports

- Profit & Loss
- Balance Sheet
- Cash Flow

Constraints

- No transaction bypasses accounting
- All entries reversible via controlled processes

6.8 CRM

Capabilities

- Customer profiles with purchase history
- Customer tagging (e.g., VIP)

Requirements

- Fast lookup at POS
- Transaction linkage to customer records
- Future-ready for loyalty integration

6.9 AI & Automation Layer (Phase 3)

OCR Processing

- Extract structured data from invoices
- Feed into accounting workflows

Anomaly Detection

- Flag:
 - Unusual transaction volumes
 - Inventory mismatches
 - Financial inconsistencies

Natural Language Interface

- Query system using plain language

7. System Architecture Principles

- Modular architecture (domain-driven services)
- Event-driven design (all actions trigger events)
- Single source of truth for financial data
- Audit-first design (every action is logged)

8. Non-Functional Requirements

Performance

- POS transaction latency < 2 seconds
- API latency < 500ms (p95)

Reliability

- 99.9% uptime
- Offline-capable POS

Security

- JWT authentication
- RBAC enforcement
- Encryption (TLS in transit, AES at rest)

Data Integrity

- Strong consistency for financial data
- Eventual consistency for analytics

9. Data & Integration Requirements

Integrations

- Payment gateways (Paystack, Flutterwave)
- OCR and AI services

Data Requirements

- Daily backups
- Immutable audit logs
- Versioning for financial records

10. Key User Flows

POS Flow

1. Add items to cart
2. Attach customer
3. Select payment method
4. Confirm transaction
5. Inventory deducted
6. Accounting updated

Requisition Flow

1. Create requisition
2. Submit for approval
3. MD approves/rejects
4. Inventory reserved
5. Shipment initiated
6. Receiving branch confirms

11. Success Metrics

Operational

- Inventory accuracy rate
- Requisition fulfillment time

Financial

- Reduction in reconciliation time
- Accuracy of financial reports

Product

- Daily active users
- Transaction volume
- Stockout reduction rate

12. Risks & Mitigations

Risk	Mitigation
Inventory inconsistency	Event-driven tracking + audit logs

Approval delays

Defined SLA for approvals

Data errors

Validation rules + audit trail

Offline conflicts

Conflict resolution via timestamps

13. Open Decisions

- Offline-first vs offline-capable architecture
- Approval hierarchy depth
- Notification channels (SMS, email, in-app)

14. Execution Notes

- Phase 1 must prioritize transaction integrity and stock visibility
- Requisition system should be introduced early for operational efficiency
- Financial accuracy must not be compromised for speed